

EXERCISE 4 SQL JOINS

Question 1 - Show every user who has a subscription. Match users to subscriptions.

--QUESTION 1 Show every user who has a subscription. Match users to subscriptions.

SELECT A.user_id,
..... user_name,
..... B.subscription_id,
..... start_date
FROM exercise4.joins.users AS A
INNER JOIN exercise4.joins.subscriptions AS B
ON A.user_id=B.user_id;

Add parameter

Table

+

Q

Y

⌵

📄

🔒

^

✕

	1 ² ₃ user_id	A ^B _C user_name	1 ² ₃ subscription_id	📅 start_date
1	1	Nomvula	503	2026-03-10
2	2	David	502	2026-02-01
3	3	Anele	505	2026-04-05
4	1	Nomvula	501	2026-01-15

⬇

▼

4 rows

2.540s runtime

[📊 See performance \(1\)](#)

✓ Checked for improvements

Refreshed 3m ago

Question 2- Show every subscription with its matching plan name and monthly

```
--QUESTION 2 Show every subscription with its matching plan name and monthly price
SELECT A.subscription_id,
       A.user_id,
       plan_name,
       monthly_price
FROM exercise4.joins.subscriptions AS A
INNER JOIN exercise4.joins.plans AS B
ON A.plan_id=B.plan_id;
```

Add parameter

Table ▼ + Q Y ⌵ [

	¹ ₃ subscription_id	¹ ₃ user_id	^A _C plan_name	¹ ₃ monthly_price
1	501	1	Basic	79
2	504	6	Standard	129
3	503	1	Premium	199
4	505	3	Family	249
5	502	2	Standard	129

⬇ ▼ 5 rows | 0.070s runtime [See performance \(1\)](#) ✓ Checked for improvements

Question 3- Show every viewing session that has a matching show. Include the show title and genre

```
--QUESTION 3 Show every viewing session that has a matching show. Include the show title and genre
SELECT A.session_id,
       A.user_id,
       B.show_title,
       B.genre,
       watch_minutes
FROM exercise4.joins.viewing_sessions AS A
INNER JOIN exercise4.joins.shows AS B
ON A.show_id=B.show_id;
```

Add parameter

Table +

🔍 ⚙️ 📄 🔄

	¹ ₃ session_id	¹ ₃ user_id	^A _C show_title	^A _C genre	¹ ₃ watch_minutes
1	904	7	Comedy Hour	Comedy	20
2	903	1	Crime Time	Drama	60
3	902	2	Tech Tales	Documentary	30
4	901	1	Comedy Hour	Comedy	45

Question 4- Show every viewing session with the user who watched it. Only show sessions with a matching user

```
--QUESTION 4 Show every viewing session with the user who watched it. Only show sessions with a matching user
SELECT B.user_name,
       country,
       A.session_id,
       A.show_id,
       watch_minutes
FROM exercise4.joins.viewing_sessions AS A
INNER JOIN exercise4.joins.users AS B
ON A.user_id=B.user_id;
```

Add parameter

Table  



	^A _C user_name	^A _C country	¹ ₃ session_id	¹ ₃ show_id	¹ ₃ watch_minutes
1	Nomvula	Johannesburg	903	702	60
2	David	Capetown	902	703	30
3	Anele	Durban	905	705	90
4	Nomvula	Johannesburg	901	701	45

  4 rows | 0.131s runtime  [See performance \(1\)](#)  Checked for improvements

Question 5- Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan.

```
--QUESTION 5 Show users along with their subscriptions, the plan name, and the price. Use only users who have both a
subscription and a valid plan
SELECT A.user_name,
       .....country,
       .....C.plan_name,
       .....monthly_price,
       .....start_date
FROM exercise4.joins.users AS A
INNER JOIN exercise4.joins.subscriptions AS B
ON A.user_id=B.user_id
INNER JOIN exercise4.joins.plans AS C
ON B.plan_id=C.plan_id;
```

Add parameter

Table  

    

	 user_name	 country	 plan_name	 monthly_price	 start_date
1	Nomvula	Johannesburg	Premium	199	2026-03-10
2	David	Capetown	Standard	129	2026-02-01
3	Anele	Durban	Family	249	2026-04-05
4	Nomvula	Johannesburg	Basic	79	2026-01-15

Question 6- Show every user and any subscriptions they have. Users without subscriptions must still appear

```
--QUESTION 6 Show every user and any subscriptions they have. Users without subscriptions must still appear
SELECT A.user_id,
       user_name,
       B.subscription_id,
       start_date
FROM exercise4.joins.users AS A
LEFT JOIN exercise4.joins.subscriptions AS B
ON A.user_id=B.user_id;
```

Add parameter

Table					+	🔍	🔧	📄
	1 ² ₃ user_id	A ^B _C user_name	1 ² ₃ subscription_id	📅 start_date				
1	1	Nomvula	503	2026-03-10				
2	2	David	502	2026-02-01				
3	3	Anele	505	2026-04-05				
4	4	Kabelo	null	null				
5	5	Lerato	null	null				
6	1	Nomvula	501	2026-01-15				
⬇	6 rows	16.906s runtime	📊 See performance (1)	✓ Checked for improvements				Re

Question 7- Show every plan and the subscriptions on it. Plans with no subscribers must still appear

--QUESTION 7 Show every plan and the subscriptions on it. Plans with no subscribers must still appear

SELECT

A.plan_id,

.....plan_name,

.....subscription_id,

.....user_id

FROM

exercise4.joins.plans

AS

A

LEFT JOIN

exercise4.joins.subscriptions

AS

B

ON

A.plan_id=B.plan_id;

Add parameter

Table

+

Q

Y

⌵

□

	plan_id	plan_name	subscription_id	user_id
1	10	Basic	501	1
2	11	Standard	504	6
3	12	Premium	503	1
4	13	Family	505	3
5	14	Mobile	null	null
6	11	Standard	502	2

↓

6 rows

0.215s runtime

[See performance \(1\)](#)

✓ Checked for improvements

Question 8- Show every show and any viewing sessions on it. Shows that were never watched must still appear

```
--QUESTION 8 Show every show and any viewing sessions on it. Shows that were never watched must still appear.
SELECT A.show_id,
       show_title,
       B.session_id,
       watch_minutes
FROM exercise4.joins.shows AS A
LEFT JOIN exercise4.joins.viewing_sessions AS B
ON A.show_id=B.show_id;
```

Add parameter

Table +					Q	Y	I	□
	1 ² ₃ show_id	A ^B _C show_title	1 ² ₃ session_id	1 ² ₃ watch_minutes				
1	701	Comedy Hour	904	20				
2	702	Crime Time	903	60				
3	703	Tech Tales	902	30				
4	704	Cooking Lab	null	null				
5	706	Wild Earth	null	null				
6	701	Comedy Hour	901	45				

↓

6 rows | 1.821s runtime [See performance \(1\)](#) ✓ Checked for improvements

Question 9- Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details).

```
--QUESTION 9
--Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear
(with NULL user details).
SELECT A.session_id,
       show_id,
       watch_minutes,
       B.user_id,
       user_name
FROM exercise4.joins.viewing_sessions AS A
LEFT JOIN exercise4.joins.users AS B
ON A.user_id=B.user_id;
```

Add parameter

Table +						🔍 🔧 📄 📁 ⬆ ✕	
	¹ ₂ session_id	¹ ₂ show_id	¹ ₂ watch_minutes	¹ ₂ user_id	^A _C user_name		
1	901	701	45	1	Nomvula		
2	902	703	30	2	David		
3	903	702	60	1	Nomvula		
4	904	701	20	null	null		
5	905	705	90	3	Anele		

⬇ ⌵ 5 rows | 0.818s runtime [📊 See performance \(1\)](#) ✓ Checked for improvements Refreshed 1m ago

Question 10- Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear.

--QUESTION 10 Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear.

SELECT A.user_name,
country,
C.plan_name,
C.monthly_price
FROM exercise4.joins.users AS A
LEFT JOIN exercise4.joins.subscriptions AS B
ON A.user_id=B.user_id
LEFT JOIN exercise4.joins.plans AS C
ON B.plan_id=C.plan_id

Add parameter

Table +

Q

Y

^

X

	A ^B _C user_name	A ^B _C country	A ^B _C plan_name	1 ² ₃ monthly_price
1	Nomvula	Johannesburg	Premium	199
2	David	Capetown	Standard	129
3	Anele	Durban	Family	249
4	Kabelo	Pretoria	null	null
5	Lerato	Port Elizabeth	null	null
6	Nomvula	Johannesburg	Basic	79

↓

6 rows

0.065s runtime

[See performance \(1\)](#)

✓ Checked for improvements

Refreshed Now

Question 11- Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist.

--QUESTION 11 Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist.
SELECT A.user_id,
A.user_name,
subscription_id,
start_date
FROM exercise4.joins.users AS A
FULL JOIN exercise4.joins.subscriptions AS B
ON A.user_id=B.user_id;

Add parameter

Table +

7 rows

0.899s runtime

[See performance \(1\)](#)

✓ Checked for improvements

Refreshed 2m ago

	user_id	user_name	subscription_id	start_date
2	1	Nomvula	503	2026-03-10
3	3	Anele	505	2026-04-05
4	2	David	502	2026-02-01
5	4	Kabelo	null	null
6	1	Nomvula	501	2026-01-15
7	null	null	504	2026-03-20

Question 12- Show every plan and every subscription, including plans without subscribers AND any subscription referencing a plan that does not exist

--QUESTION 12 Show every plan and every subscription, including plans without subscribers AND any subscription referencing a plan that does not exist.

```
SELECT DISTINCT plans.plan_id,
               plan_name,
               subscription_id,
               user_id
FROM exercise4.joins.subscriptions
FULL JOIN exercise4.joins.plans
ON subscriptions.plan_id=plans.plan_id;
```

Add parameter

Table +

🔍 ⚙️ 📄 🗑️ ⬆️ ✕

	plan_id	plan_name	subscription_id	user_id
1	11	Standard	504	6
2	11	Standard	502	2
3	12	Premium	503	1
4	10	Basic	501	1
5	13	Family	505	3
6	14	Mobile	null	null

⬇️ ⌵

6 rows | 0.777s runtime | [See performance \(1\)](#) | ✓ Checked for improvements

Refreshed Now

Question 13 - Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist.

--QUESTION 13 Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist.

```
SELECT shows.show_id,  
       show_title,  
       session_id,  
       watch_minutes  
FROM exercise4.joins.shows  
FULL JOIN exercise4.joins.viewing_sessions  
ON shows.show_id=viewing_sessions.show_id
```

Add parameter

Table +					Q F I E ^ X				
	show_id	show_title	session_id	watch_minutes					
1	704	Cooking Lab	null	null					
2	703	Tech Tales	902	30					
3	701	Comedy Hour	904	20					
4	702	Crime Time	903	60					
5	706	Wild Earth	null	null					
6	701	Comedy Hour	901	45					
7 rows 0.647s runtime See performance (1) Checked for improvements					Refreshed 2m ago				

Question 14 - Show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist.

--QUESTION 14 Show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist.
SELECT users.user_id,
 user_name,
 session_id,
 show_id,
 watch_minutes
FROM exercise4.joins.users
FULL JOIN exercise4.joins.viewing_sessions
ON users.user_id=viewing_sessions.user_id

--QUESTION 15 Show every user, every subscription, and every plan in one query – using FULL OUTER JOIN throughout. This is

Add parameter

Table +

Q

F

I

□

⌕

^

×

	123 user_id	A0 user_name	123 session_id	123 show_id	123 watch_minutes
1	1	Nomvula	901	701	45
2	1	Nomvula	903	702	60
3	3	Anele	905	705	90
4	2	David	902	703	30
5	null	null	904	701	20
6	4	Kabelo	null	null	null
7	5	Lerato	null	null	null

↓

7 rows

0.191s runtime

[See performance \(1\)](#)

✓ Checked for improvements

Refreshed Now

Question 15- Show every user, every subscription, and every plan in one query — using FULL OUTER JOIN throughout. This is the hardest question — get all gaps visible at once.

```
--QUESTION 15 Show every user, every subscription, and every plan in one query – using FULL OUTER JOIN throughout. This is the hardest question – get all gaps visible at once.
```

```
SELECT users.user_id,  
       user_name,  
       subscription_id,  
       plans.plan_id,  
       plan_name  
FROM exercise4.joins.users  
FULL JOIN exercise4.joins.subscriptions  
ON users.user_id=subscriptions.user_id  
FULL JOIN exercise4.joins.plans  
ON subscriptions.plan_id=plans.plan_id
```

Add parameter

	¹ ₃ user_id	^A _C user_name	¹ ₃ subscription_id	¹ ₃ plan_id	^A _C plan_name
1	1	Nomvula	503	12	Premium
2	1	Nomvula	501	10	Basic
3	3	Anele	505	13	Family
4	5	Lerato	null	null	null
5	4	Kabelo	null	null	null
6	2	David	502	11	Standard
7	null	null	504	11	Standard
8	null	null	null	14	Mobile

 

8 rows | 0.064s runtime [See performance \(1\)](#)  Checked for improvements

Refreshed 3m ago

Bonus Challenge -

Question 1- Which users have not subscribed to any plan?

--Bonus Challenge
--Question Which users have not subscribed to any plan?
SELECT users.user_id,
.....user_name
FROM exercise4.joins.users
LEFT JOIN exercise4.joins.subscriptions
ON users.user_id=subscriptions.user_id
WHERE subscription_id IS NULL;

Add parameter

Table +

🔍 🏠 📄 📁 📂 ^ ✕

	🔗 user_id	🔗 user_name
1	4	Kabelo
2	5	Lerato

Question 2- Which subscriptions reference users that do not exist in the users table?

```
--QUESTION 2 Which subscriptions reference users that do not exist in the users table?  
SELECT subscription_id,  
       subscriptions.user_id  
FROM exercise4.joins.subscriptions  
LEFT JOIN exercise4.joins.users  
ON subscriptions.user_id=users.user_id  
WHERE users.user_id IS NULL;
```

Add parameter

Table ▾ +

Q Y I □ ⚙ ^ X

	¹ ₃ subscription_id	¹ ₃ user_id
1	504	6

Question 3- Which shows have never been watched?

```
FROM exercise4.joins.subscriptions
LEFT JOIN exercise4.joins.users
ON subscriptions.user_id=users.user_id
WHERE users.user_id IS NULL;

--QUESTION 3 Which shows have never been watched?
SELECT shows.show_id,
       show_title
FROM exercise4.joins.shows
LEFT JOIN exercise4.joins.viewing_sessions
ON shows.show_id=viewing_sessions.show_id
WHERE session_id IS NULL;
```

Add parameter

Table +

Q

Y

⌵

⌶

⌵

⌵

⌵

⌵

	¹ ₃ show_id	^A _C show_title
1	704	Cooking Lab
2	706	Wild Earth

Question 4- Which viewing sessions reference shows that do not exist?

--QUESTION 4 Which viewing sessions reference shows that do not exist?
SELECT viewing_sessions.session_id,
viewing_sessions.show_id
FROM exercise4.joins.viewing_sessions
LEFT JOIN exercise4.joins.shows
ON viewing_sessions.show_id=shows.show_id
WHERE shows.show_id IS NULL

Add parameter

Table +

Q

Y

⌵

⌶

⌵

^

×

	1 session_id	1 show_id
1	905	705

Question 5- Which plans have no subscribers?

▶ Run selected (1000) ✓ Just now (1s) workspace default

Edit

--QUESTION 5 Which plans have no subscribers?

SELECT DISTINCT
.....plans.plan_id,
.....plan_name
FROM exercise4.joins.plans
LEFT JOIN exercise4.joins.subscriptions
....ON plans.plan_id = subscriptions.plan_id
WHERE subscription_id IS NULL;

Add parameter

Table +

plan_idplan_name

114Mobile

	¹ ₃ plan_id	^A _C plan_name
1	14	Mobile

